
1. Déclarez une structure FileS pour une file statique avec :

- un tableau d'entiers de taille MAX
- deux indices debut et queue

2. Implémentez les fonctions suivantes :

- initialiserFileS()
- enfilerS()
- defilerS()
- estVideS()
- estPleineS()
- afficherFileS()

Utilisez la file statique de l'exercice 1 pour simuler un guichet bancaire avec des clients qui arrivent (enfiler) et sont servis (défiler). Chaque client est représenté par un numéro d'identifiant (int). Affichez la file à chaque opération.

1. Déclarez les structures Elem et FileD.

2. Implémentez les fonctions de base :

- initFileD()
- enfilerD()
- defilerD()
- estVideD()
- afficherFileD()
- libererFileD()

3. Testez avec un menu simple.

Chaque élément de la file contient un enregistrement Client :

```
typedef struct {  
    int id;  
    char nom[30];  
} Client;
```

1. Modifiez les fonctions précédentes pour gérer des Client au lieu de simples entiers.
2. Affichez les informations des clients au fur et à mesure.

1. Ajoutez deux fonctions à votre gestion de file dynamique :

- sauvegarderFileDansFichier(FileD* file, const char* nomFichier)
- chargerFileDepuisFichier(FileD* file, const char* nomFichier)

2. Testez avec un fichier texte contenant plusieurs entiers ou objets Client.

Créez un programme complet de gestion d'une file d'attente dynamique :

- Interface utilisateur en console (menu interactif)
- Gestion des clients (struct Client avec ID, nom, type de service)
- Sauvegarde et chargement de la file
- Statistiques (nombre de clients, client en tête, client en queue)
- Bonus : temps moyen d'attente simulé